

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: Operating Systems

NAME: G. Yashmita

COURSE CODE: CSA0401

REG NO: 192511032

COURSE OUTCOME COVERED

CO3: Evaluate memory management mechanisms such as linking, paging, and segmentation to determine their effectiveness for different system requirements

Assessment Tool Weightage: 40%

QUESTIONS: 5

Total Marks: 50

Experiment Questions:

- 1.Design and implement a program to simulate the translation of a logical address into a physical address using base and limit registers. The program should accept logical address as input, check for validity, and compute the corresponding physical address. Display appropriate messages for valid and invalid addresses.
- 2.Write a program to demonstrate the paging memory management scheme. The program should divide memory into fixed-size pages and frames, accept page number and offset as input, and compute the corresponding physical address using a page table.
- 3.Develop a program to simulate a page table. The program should allow the user to input page table entries and logical addresses, then map them to physical addresses. Clearly show how page numbers are translated into frame numbers.
- 4.Implement a program to simulate the working of a Translation Lookaside Buffer (TLB). The program should demonstrate how TLB improves memory access time by calculating the effective memory access time with and without TLB.
- 5.Write a program to illustrate memory protection using base and limit registers. The program should verify whether a given logical address lies within the valid range and prevent unauthorized access.

RUBRICS FOR CALCULATION:

Criteria	Weightage (Marks)	Level 4 – Exemplary (Full Marks)	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Conceptual Understanding	35 Marks	Demonstrates clear, complete, and in-depth understanding of OS concepts (types, structure, system calls)	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answers	25 Marks	All answers are correct, precise, and relevant	Mostly correct with minor errors	Partially correct with noticeable errors	Mostly incorrect or irrelevant
Application of Knowledge	15 Marks	Effectively applies concepts to scenarios/questions	Applies with minor mistakes	Limited or partial application	No meaningful application
Completeness of Response	15 Marks	All parts of each question are fully answered	Most parts covered with small omissions	Some parts missing	Incomplete answers
Clarity & Presentation	10 Marks	Clear, well-structured, and neatly presented answers	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand

Question 1: Logical Address to Physical Address using Base and Limit Registers

Aim

To design and implement a program to translate a logical address into a physical address using **base and limit registers**, and to check whether the logical address is valid.

Concept

- **Base register** stores the starting physical address of a process.
- **Limit register** stores the maximum valid logical address/range.
- If the logical address is within the limit:

$$\text{Physical Address} = \text{Base Address} + \text{Logical Address}$$

- Otherwise, the address is invalid and access is denied.

Algorithm

1. Read the base address.
2. Read the limit value.
3. Read the logical address.
4. Check whether logical address < limit.
5. If valid, calculate:
physical address = base + logical address.
6. Otherwise, display an invalid-address message.

C Program

```
#include <stdio.h>
```

```
int main()
{
    int base, limit, logical, physical;

    printf("Enter Base Address: ");
    scanf("%d", &base);

    printf("Enter Limit: ");
    scanf("%d", &limit);

    printf("Enter Logical Address: ");
    scanf("%d", &logical);

    if (logical >= 0 && logical < limit)
    {
        physical = base + logical;

        printf("\nLogical Address : %d", logical);
```

```

        printf("\nPhysical Address: %d", physical);
        printf("\nAddress is VALID.\n");
    }
    else
    {
        printf("\nLogical Address : %d", logical);
        printf("\nAddress is INVALID. Access denied.\n");
    }

    return 0;
}

```

Sample Output

Enter Base Address: 1000
 Enter Limit: 500
 Enter Logical Address: 250

Logical Address : 250
 Physical Address: 1250
 Address is VALID.

Invalid Address

Enter Base Address: 1000
 Enter Limit: 500
 Enter Logical Address: 600

Logical Address : 600
 Address is INVALID. Access denied.

```

main.c
1  #include <stdio.h>
2
3  int main()
4  {
5      int base, limit, logical, physical;
6
7      printf("Enter Base Address: ");
8      scanf("%d", &base);
9
10     printf("Enter Limit: ");
11     scanf("%d", &limit);
12
13     printf("Enter Logical Address: ");
14     scanf("%d", &logical);
15
16     if (logical >= 0 && logical < limit)
17     {
18         physical = base + logical;
19
20         printf("\nLogical Address : %d", logical);
21         printf("\nPhysical Address: %d", physical);
22         printf("\nAddress is VALID.\n");
23     }
24     else
25     {
26         printf("\nLogical Address : %d", logical);
27         printf("\nAddress is INVALID. Access denied.\n");
28     }
29
30     return 0;
31 }

```

Enter Base Address: 1000
 Enter Limit: 500
 Enter Logical Address: 50
 Logical Address : 50
 Physical Address: 1050
 Address is VALID.

```
Enter Base Address: 1000
Enter Limit: 500
Enter Logical Address: 600

Logical Address : 600
Address is INVALID. Access denied.

...Program finished with exit code 0
```

Result

Thus, the logical address was successfully translated into a physical address using the base and limit registers, and invalid addresses were prevented.

Question 2: Paging Memory Management Scheme

Aim

To implement a program to demonstrate the **paging memory management scheme** by dividing memory into fixed-size pages and frames and translating a logical address into a physical address using a page table.

Concept

Paging divides:

- Logical memory → **Pages**
- Physical memory → **Frames**

A logical address consists of:

Logical Address = Page Number + Offset

The page table maps:

Page Number → Frame Number

Physical address is calculated as:

Physical Address = Frame Number × Page Size + Offset

Algorithm

1. Read the number of pages.
2. Read the page size.
3. Enter the frame number for each page.
4. Read the page number and offset.
5. Check whether the page number is valid.
6. Check whether the offset is within the page size.
7. Obtain the frame number from the page table.

8. Calculate physical address.
9. Display the result.

C Program

```
#include <stdio.h>

int main()
{
    int pages, pageSize;
    int pageTable[100];
    int page, offset, frame, physical;
    int i;

    printf("Enter number of pages: ");
    scanf("%d", &pages);

    printf("Enter page size: ");
    scanf("%d", &pageSize);

    printf("\nEnter frame number for each page:\n");

    for (i = 0; i < pages; i++)
    {
        printf("Page %d -> Frame: ", i);
        scanf("%d", &pageTable[i]);
    }

    printf("\nEnter Page Number: ");
    scanf("%d", &page);

    printf("Enter Offset: ");
    scanf("%d", &offset);

    if (page < 0 || page >= pages)
    {
        printf("\nInvalid Page Number.\n");
    }
    else if (offset < 0 || offset >= pageSize)
    {
        printf("\nInvalid Offset.\n");
    }
    else
    {
        frame = pageTable[page];

        physical = frame * pageSize + offset;

        printf("\nPage Number    : %d", page);
        printf("\nOffset          : %d", offset);
    }
}
```

```

        printf("\nFrame Number   : %d", frame);
        printf("\nPhysical Address: %d\n", physical);
    }

    return 0;
}

```

Sample Output

Enter number of pages: 4
Enter page size: 100

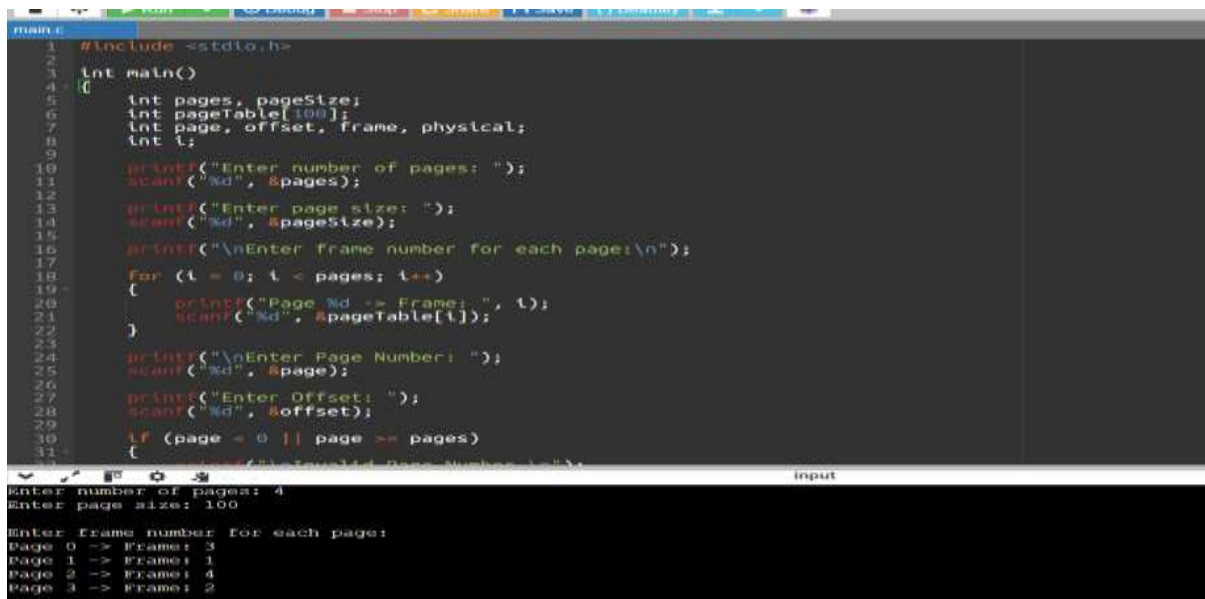
Enter frame number for each page:
Page 0 -> Frame: 3
Page 1 -> Frame: 1
Page 2 -> Frame: 4
Page 3 -> Frame: 2

Enter Page Number: 2
Enter Offset: 25
Page Number: 2
Offset: 25
Frame Number: 4
Physical Address: 425

Because:

Frame = 4
Page Size = 100
Offset = 25

Physical Address = $(4 \times 100) + 25$
= 425



```

main.c
1  #include <stdio.h>
2
3  int main()
4  {
5      int pages, pageSize;
6      int pageTable[100];
7      int page, offset, frame, physical;
8      int i;
9
10     printf("Enter number of pages: ");
11     scanf("%d", &pages);
12
13     printf("Enter page size: ");
14     scanf("%d", &pageSize);
15
16     printf("\nEnter frame number for each page:\n");
17
18     for (i = 0; i < pages; i++)
19     {
20         printf("Page %d -> Frame: ", i);
21         scanf("%d", &pageTable[i]);
22     }
23
24     printf("\nEnter Page Number: ");
25     scanf("%d", &page);
26
27     printf("Enter Offset: ");
28     scanf("%d", &offset);
29
30     if (page < 0 || page >= pages)
31     {
32         printf("Invalid Page Number: %d\n", page);
33     }
34
35     frame = pageTable[page];
36     physical = (frame * pageSize) + offset;
37
38     printf("\nFrame Number: %d\n", frame);
39     printf("\nPhysical Address: %d\n", physical);
40
41     return 0;
42 }

```

input

Enter number of pages: 4
Enter page size: 100

Enter frame number for each page:
Page 0 -> Frame: 3
Page 1 -> Frame: 1
Page 2 -> Frame: 4
Page 3 -> Frame: 2

```
Enter Offset: 25
Page Number   : 2
Offset        : 25
Frame Number   : 4
Physical Address: 425
...Program finished with exit code 0
```

Result

Thus, paging was successfully implemented using a page table, and the logical address was translated into its corresponding physical address.

Question 3: Simulate a Page Table

Aim

To develop a program to simulate a page table that accepts page table entries and logical addresses and maps page numbers to frame numbers.

Concept

A **page table** maintains the mapping between pages in logical memory and frames in physical memory.

For example:

Page Number	Frame Number
0	5
1	2
2	7
3	1

If the logical address contains:

Page = 2

Offset = 30

Then:

Page 2 → Frame 7

Physical Address = $(7 \times \text{Page Size}) + 30$

Algorithm

1. Read the number of pages.
2. Read the page size.

3. Enter the frame number corresponding to each page.
4. Display the complete page table.
5. Enter a logical page number and offset.
6. Search the page table.
7. Obtain the corresponding frame number.
8. Calculate the physical address.
9. Display the mapping.

C Program

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    int pageTable[100];
```

```
    int n, pageSize;
```

```
    int page, offset, frame, physical;
```

```
    int i;
```

```
    printf("Enter number of pages: ");
```

```
    scanf("%d", &n);
```

```
    printf("Enter page size: ");
```

```
    scanf("%d", &pageSize);
```

```
    printf("\nEnter page table entries:\n");
```

```
    for (i = 0; i < n; i++)
```

```
    {
```

```
        printf("Frame number for Page %d: ", i);
```

```
        scanf("%d", &pageTable[i]);
```

```
    }
```

```
    printf("\nPAGE TABLE\n");
```

```
    printf("-----\n");
```

```
    printf("Page Number\tFrame Number\n");
```

```
    printf("-----\n");
```

```
    for (i = 0; i < n; i++)
```

```
    {
```

```
        printf("%d\t\t%d\n", i, pageTable[i]);
```

```
    }
```

```
    printf("\nEnter Page Number: ");
```

```
    scanf("%d", &page);
```

```
    printf("Enter Offset: ");
```

```
    scanf("%d", &offset);
```

```
    if (page < 0 || page >= n)
```

```

    {
        printf("\nInvalid Page Number.\n");
    }
    else if (offset < 0 || offset >= pageSize)
    {
        printf("\nInvalid Offset.\n");
    }
    else
    {
        frame = pageTable[page];

        physical = frame * pageSize + offset;

        printf("\nPage %d is mapped to Frame %d.", page, frame);
        printf("\nPhysical Address = %d\n", physical);
    }

    return 0;
}

```

Sample Output

Enter number of pages: 4

Enter page size: 100

Enter page table entries:

Frame number for Page 0: 5

Frame number for Page 1: 2

Frame number for Page 2: 7

Frame number for Page 3: 1

PAGE TABLE

Page Number	Frame Number
0	5
1	2
2	7
3	1

Enter Page Number: 2

Enter Offset: 30

Page 2 is mapped to Frame 7.

Physical Address = 730

```
1 #include <stdio.h>
2
3 int main()
4 {
5     int pageTable[100];
6     int n, pageSize;
7     int page, offset, frame, physical;
8     int i;
9
10    printf("Enter number of pages: ");
11    scanf("%d", &n);
12
13    printf("Enter page size: ");
14    scanf("%d", &pageSize);
15
16    printf("\nEnter page table entries:\n");
17    for (i = 0; i < n; i++)
18    {
19        printf("Frame number for Page %d: ", i);
20        scanf("%d", &pageTable[i]);
21    }
22
23    printf("\nPAGE TABLE\n");
24    printf("-----\n");
25    printf("Page Number\tFrame Number\n");
26    printf("-----\n");
27
28    for (i = 0; i < n; i++)
29    {
30        printf("%d\t%d\n", i, pageTable[i]);
31    }
32
33    printf("\nEnter Page Number: ");
34    scanf("%d", &page);
35    printf("Enter Offset: ");
36    scanf("%d", &offset);
37
38    printf("Page %d is mapped to Frame %d.\n", page, pageTable[page]);
39    printf("Physical Address = %d\n", pageTable[page] * pageSize + offset);
40
41    printf("\n...Program finished with exit code 0\n");
42    printf("Press ENTER to exit console.\n");
43}
```

input

Enter number of pages: 4
Enter page size: 100
Enter page table entries:
Frame number for Page 0: 5
Frame number for Page 1: 2
Frame number for Page 2: 7
Frame number for Page 3: 1

Page Number	Frame Number
0	5
1	2
2	7
3	1

Enter Page Number: 2
Enter Offset: 30
Page 2 is mapped to Frame 7.
Physical Address = 730

...Program finished with exit code 0
Press ENTER to exit console.

Result

Thus, the page table was successfully simulated, and page numbers were mapped to their corresponding frame numbers to obtain physical addresses.

Question 4: Translation Lookaside Buffer (TLB)

Aim

To implement a program to simulate a **Translation Lookaside Buffer (TLB)** and demonstrate how TLB improves memory access time compared with memory access without TLB.

Concept

A **TLB** is a small, fast cache that stores recently used page-to-frame mappings.

Without TLB

For every memory reference:

Memory access for page table
+
Memory access for actual data

Therefore:

EAT without TLB = Page Table Access Time + Memory Access Time

With TLB

If the page number is found in the TLB (**TLB hit**):

EAT = TLB Access Time + Memory Access Time

If the page number is not found (**TLB miss**):

EAT = TLB Access Time + Page Table Access Time + Memory Access Time

C Program

```
#include <stdio.h>

int main()
{
    float tlbTime, memoryTime;
    float hitRatio;
    float hitTime, missTime, eatWithTLB, eatWithoutTLB;

    printf("Enter TLB access time: ");
    scanf("%f", &tlbTime);

    printf("Enter Memory access time: ");
    scanf("%f", &memoryTime);

    printf("Enter TLB hit ratio (0 to 1): ");
    scanf("%f", &hitRatio);

    hitTime = tlbTime + memoryTime;

    missTime = tlbTime + memoryTime + memoryTime;

    eatWithTLB = (hitRatio * hitTime) +
        ((1 - hitRatio) * missTime);

    eatWithoutTLB = 2 * memoryTime;

    printf("\nTLB Hit Time      = %.2f units", hitTime);
    printf("\nTLB Miss Time       = %.2f units", missTime);
```

```

printf("\nEAT Without TLB   = %.2f units", eatWithoutTLB);
printf("\nEAT With TLB      = %.2f units\n", eatWithTLB);

if (eatWithTLB < eatWithoutTLB)
    printf("\nTLB improves memory access time.\n");
else
    printf("\nTLB does not improve memory access time.\n");

return 0;
}

```

Sample Output

Enter TLB access time: 10
Enter Memory access time: 100
Enter TLB hit ratio (0 to 1): 0.8

TLB Hit Time = 110.00 units
TLB Miss Time = 210.00 units
EAT Without TLB = 200.00 units
EAT With TLB = 130.00 units

TLB improves memory access time.

Calculation

For hit:

TLB access = 10
Memory access = 100

Hit time = 10 + 100
 = 110

For miss:

Miss time = 10 + 100 + 100
 = 210

Effective access time:

$EAT = (0.8 \times 110) + (0.2 \times 210)$
 = 88 + 42
 = 130

Without TLB:

$EAT = 100 + 100$
 = 200

Therefore:

With TLB = 130

Without TLB = 200

So, TLB reduces the effective memory access time.

```
1  #include <stdio.h>
2
3  int main()
4  {
5      float tlbTime, memoryTime;
6      float hitRatio;
7      float hitTime, missTime, eatWithTLB, eatWithoutTLB;
8
9      printf("Enter TLB access time: ");
10     scanf("%f", &tlbTime);
11
12     printf("Enter Memory access time: ");
13     scanf("%f", &memoryTime);
14
15     printf("Enter TLB hit ratio (0 to 1): ");
16     scanf("%f", &hitRatio);
17
18     hitTime = tlbTime + memoryTime;
19
20     missTime = tlbTime + memoryTime + memoryTime;
21
22     eatWithTLB = (hitRatio * hitTime) +
23                 ((1 - hitRatio) * missTime);
24
25     eatWithoutTLB = 2 * memoryTime;
26
27     printf("\nTLB Hit Time      = %.2f units", hitTime);
28     printf("\nTLB Miss Time       = %.2f units", missTime);
29     printf("\nEAT Without TLB      = %.2f units", eatWithoutTLB);
30     printf("\nEAT With TLB        = %.2f units\n", eatWithTLB);
31
32     if (eatWithTLB < eatWithoutTLB)
33     {
34         printf("TLB improves memory access time.\n");
35     }
36
37     ...Program finished with exit code 0
38     Press ENTER to exit console.
```

Result

Thus, the TLB mechanism was successfully simulated, and it was demonstrated that a higher TLB hit ratio improves memory access performance.

Question 5: Memory Protection using Base and Limit Registers

Aim

To implement a program to illustrate **memory protection using base and limit registers** and prevent unauthorized access to memory.

Concept

Memory protection ensures that a process accesses only its assigned memory region.

If:

Base = starting address
Limit = size of allocated memory

then the valid physical address range is:

$$\text{Base} \leq \text{Physical Address} < \text{Base} + \text{Limit}$$

For a logical address:

$$\text{Physical Address} = \text{Base} + \text{Logical Address}$$

The logical address is valid only when:

$$0 \leq \text{Logical Address} < \text{Limit}$$

Otherwise, access is denied.

Algorithm

1. Read the base address.
2. Read the limit.
3. Read the logical address.
4. Check whether the logical address is within the limit.
5. If valid, calculate the physical address.
6. If invalid, prevent access and display an error message.

C Program

```
#include <stdio.h>

int main()
{
    int base, limit, logical, physical;

    printf("Enter Base Address: ");
    scanf("%d", &base);

    printf("Enter Limit: ");
    scanf("%d", &limit);

    printf("Enter Logical Address: ");
    scanf("%d", &logical);

    if (logical >= 0 && logical < limit)
    {
        physical = base + logical;

        printf("\nACCESS GRANTED");
        printf("\nLogical Address : %d", logical);
        printf("\nPhysical Address: %d\n", physical);
    }
}
```

```

    }
    else
    {
        printf("\nACCESS DENIED");
        printf("\nLogical Address %d is outside the valid memory range.\n",
            logical);
    }

    return 0;
}

```

Sample Output – Authorized Access

Enter Base Address: 2000
Enter Limit: 500
Enter Logical Address: 300

ACCESS GRANTED
Logical Address : 300
Physical Address: 2300

Sample Output – Unauthorized Access

Enter Base Address: 2000
Enter Limit: 500
Enter Logical Address: 700

ACCESS DENIED
Logical Address 700 is outside the valid memory range.

```

1  #include <stdio.h>
2
3  int main()
4  {
5      int base, limit, logical, physical;
6
7      printf("Enter Base Address: ");
8      scanf("%d", &base);
9
10     printf("Enter Limit: ");
11     scanf("%d", &limit);
12
13     printf("Enter Logical Address: ");
14     scanf("%d", &logical);
15
16     if (logical >= 0 && logical < limit)
17     {
18         physical = base + logical;
19
20         printf("\nACCESS GRANTED");
21         printf("\nLogical Address : %d", logical);
22         printf("\nPhysical Address: %d\n", physical);
23     }
24     else
25     {
26         printf("\nACCESS DENIED");
27         printf("\nLogical Address %d is outside the valid memory range.\n",
28             logical);
29     }
30
31     return 0;
32 }

```

Enter Base Address: 2000
Enter Limit: 500
Enter Logical Address: 300

ACCESS GRANTED
Logical Address : 300
Physical Address: 2300

...Program finished with exit code 0
Press ENTER to exit console.

Result

Thus, memory protection was successfully implemented using base and limit registers, and unauthorized memory access was prevented.